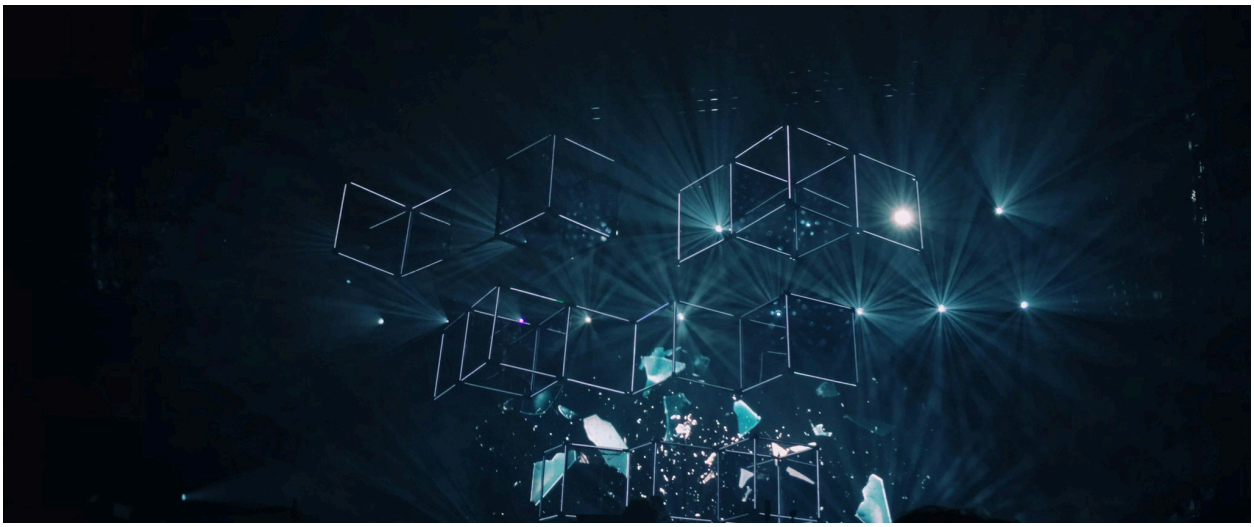

SQL-II

SQL Data Types

Numeric Data Types

- **INT/INTEGER**: Whole numbers (-2,147,483,648 to 2,147,483,647)
- **SMALLINT**: Small integers (-32,768 to 32,767)
- **BIGINT**: Large integers
- **DECIMAL(p,s)/NUMERIC(p,s)**: Fixed-point numbers
- **FLOAT**: Floating-point numbers
- **DOUBLE**: Double-precision floating-point

String Data Types

- **CHAR(n)**: Fixed-length character string
- **VARCHAR(n)**: Variable-length character string
- **TEXT**: Large text data
- **BLOB**: Binary large object

Date and Time Data Types

- **DATE**: Date (YYYY-MM-DD)
- **TIME**: Time (HH:MM:SS)
- **DATETIME**: Date and time
- **TIMESTAMP**: Timestamp with timezone
- **YEAR**: Year value

SQL Constraints

Primary Key Constraint

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

-- Composite primary key
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id)
);
```

Foreign Key Constraint

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    department_id INT,
```

```
    FOREIGN KEY (department_id) REFERENCES departments(dept_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Other Constraints

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) CHECK (price > 0),
    category VARCHAR(50) DEFAULT 'General',
    sku VARCHAR(20) UNIQUE,
    stock_quantity INT CHECK (stock_quantity >= 0)
);
```

DDL (Data Definition Language) Commands

DDL commands are used to define and modify the structure of database objects like tables, indexes, and schemas.

CREATE Command

Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...
    table_constraints
);
```

Example:

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2) CHECK (salary > 0),
    hire_date DATE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(dept_id)
);
```

ALTER Command

Syntax:

```
-- Add column
ALTER TABLE table_name ADD column_name datatype constraint;

-- Modify column
ALTER TABLE table_name MODIFY column_name datatype constraint;

-- Drop column
ALTER TABLE table_name DROP COLUMN column_name;

-- Add constraint
ALTER TABLE table_name ADD CONSTRAINT constraint_name constraint_typ
e;
```

Examples:

```
-- Add new column
ALTER TABLE employees ADD phone VARCHAR(15);

-- Modify existing column
ALTER TABLE employees MODIFY salary DECIMAL(12,2);
```

```
-- Add foreign key constraint
ALTER TABLE employees
ADD CONSTRAINT fk_dept
FOREIGN KEY (department_id) REFERENCES departments(dept_id);

-- Drop column
ALTER TABLE employees DROP COLUMN phone;
```

DROP Command

Syntax:

```
-- Drop table
DROP TABLE table_name;

-- Drop database
DROP DATABASE database_name;

-- Drop index
DROP INDEX index_name ON table_name;
```

Examples:

```
DROP TABLE temp_employees;
DROP DATABASE test_db;
DROP INDEX idx_employee_email ON employees;
```

TRUNCATE Command

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE employees;
```

DML (Data Manipulation Language) Commands

DML commands are used to manipulate data within database tables.

INSERT Command

Syntax:

```
-- Insert single row
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);

-- Insert multiple rows
INSERT INTO table_name (column1, column2, ...)
VALUES
    (value1, value2, ...),
    (value1, value2, ...);
```

Examples:

```
-- Single row insert
INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES ('John', 'Doe', 'john.doe@company.com', 50000.00, 1);

-- Multiple rows insert
INSERT INTO departments (dept_name) VALUES
    ('Human Resources'),
    ('Information Technology'),
    ('Finance');
```

UPDATE Command

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Examples:

```
-- Update single column
UPDATE employees
SET salary = 55000.00
WHERE emp_id = 1;

-- Update multiple columns
UPDATE employees
SET salary = salary * 1.10, last_modified = NOW()
WHERE department_id = 2;

-- Update with subquery
UPDATE employees
SET salary = (SELECT AVG(salary) * 1.05 FROM employees WHERE departme
nt_id = 1)
WHERE emp_id = 5;
```

DELETE Command

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Examples:

```
-- Delete specific records
DELETE FROM employees WHERE emp_id = 10;

-- Delete with multiple conditions
DELETE FROM employees
```

```
WHERE department_id = 3 AND hire_date < '2020-01-01';

-- Delete with subquery
DELETE FROM employees
WHERE department_id IN (SELECT dept_id FROM departments WHERE dept_n
ame = 'Temp');
```

Advanced Examples with Foreign Keys

```
-- Create departments table
CREATE TABLE departments (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50) NOT NULL,
    location VARCHAR(100)
);

-- Create employees table with foreign key
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    phone VARCHAR(15),
    salary DECIMAL(10,2) CHECK (salary > 0),
    hire_date DATE DEFAULT CURRENT_DATE,
    department_id INT,
    manager_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(dept_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE,
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
        ON DELETE SET NULL
);

-- Create projects table
```

```sql
CREATE TABLE projects (
    project_id INT PRIMARY KEY AUTO_INCREMENT,
    project_name VARCHAR(100) NOT NULL,
    start_date DATE,
    end_date DATE,
    budget DECIMAL(15,2),
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(dept_id)
);

-- Insert sample data
INSERT INTO departments (dept_name, location) VALUES
    ('HR', 'Building A'),
    ('IT', 'Building B'),
    ('Finance', 'Building C');

INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES
    ('John', 'Smith', 'john.smith@company.com', 60000, 2),
    ('Jane', 'Doe', 'jane.doe@company.com', 55000, 1),
    ('Mike', 'Johnson', 'mike.johnson@company.com', 70000, 2);

-- Complex queries with joins
SELECT
    e.first_name,
    e.last_name,
    d.dept_name,
    m.first_name AS manager_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.dept_id
LEFT JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > 50000;
```